Aufgabe 2: ChoreoGraph

Team-ID: 00563

Team-Name: Matthieu Dang

Bearbeiter/-innen dieser Aufgabe:
Matthieu Dang

Inhaltsverzeichnis

1 Lösungsidee **1**
1.1 Suche der gültigen Choreographien 1
1.2 Sonderfälle 2

2 Umsetzung **2**
2.1 Repräsentation der Figuren 2
2.2 Ausführen einer Figur 2
2.3 Rekursive Suche aller Choreographien 3
2.4 Verkürzung der Laufzeit 3
2.5 Sonderfälle 3
2.6 Ausführung 4

3 Werkzeuge **4**

4 Beispiele **4**

5 Quellcode **5**
5.1 Klasse für Figuren 5
5.2 Ausführen einer Figur 6
5.3 rekursive Suche aller Choreographien 6
5.4 Sonderfälle 6
5.5 Ausführen des Programmes 7

1 Lösungsidee

In dieser Aufgabe soll eine Choreographie aus den eingegebenen Figuren und der Stücklänge in Noras Notation eine gültige Choreographie gefunden werden, dessen Länge der Stücklänge entspricht, und bei der alle Tänzer wie beim Anfang stehen. Falls es mehrere gültige Choreographien gibt, soll man für bestimmte Kriterien die beste finden.

1.1 Suche der gültigen Choreographien

Die Suche aller gültigen Kombinationen erfolgt durch die Bildung aller Kombinationen an Figuren mit der richtigen Länge, bei denen alle Tänzer zur Startposition zurückkommen. Dies funktioniert durch rekursives Backtracking. Dabei fängt man mit einer leeren Liste, zu der dann später Figuren hinzugefügt werden um eine Choreographie zu bilden. Es wird dann immer jeweils eine Figur der Figurenliste der Choreographie hinzugefügt. Dies wird dann immer weiter mit der Choreographie gemacht, bis die Takte ausgehen. Falls die Choreographie gleich lang, wie das Musikstück ist, soll man prüfen ob darin die Tänzer

nach der Ausführung wieder zu ihrer Startposition kommen, falls das auch zutrifft, ist die Choreographie gültig. Falls nicht, wird sie verworfen. Wenn man an einem Ende ankommt, wird zuzükgegangen, bis wo man noch andere Figuren anstelle der letzten einfügen kann, und wird von da wiederholt.

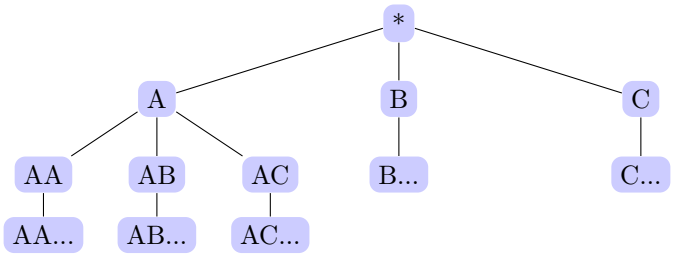


Abb. 1: Der Baum zeigt die Suche von Choreographien bei einem Beispiel mit 3 Figuren. Im Ersten Schritt wird A mit A, B und C kombiniert, um AA, AB und AC zu erhalten, dies wird immer weiter für jede dieser neunenKombinationen wiederholt. Wenn bei allen Kombinationen startend mit A die Takte ausgehen geht man zu B und anschließend C über und macht dort dasselbe.

Um zu prüfen, ob alle Tänzer am Ende, wie am Start stehen, braucht man eine Weise, wie man eine Figur mit einer bestimmten Anordnung an Tänzern ausführt und die neue Reihenfolge erhält. Diese sollte mit jeder Figur in einer Choreographie und der Endstellung der letzten Figur als Anfangsstellung ausgeführt werden, und sollte dann zum Schluss der Startstellung entsprechen.

1.2 Sonderfälle

Falls es mehr als eine gültige Choreographie, also Kombination von Firguren gibt, soll die beste Kombination in jeweils 5 Kriterien gefunden werden.

Diese wird so gefunden, dass für jeder dieser Kriterien der beste Wert zuerst herausgesucht wird, anschließend wird das erste Element mit diesem besten Wert zurückgegeben.

2 Umsetzung

2.1 Repräsentation der Figuren

Es wird zuerst eine Klasse für alle Figuren erstellt. Diese enthält die Attribute *name* für den Namen der Figur, *duration* für die Dauer der Figur in Takte und *pos_after* für die Position, die die Tänzer nach der Ausführung von der Startposition ABCDEFGHIJKLMNOP, jeder Buchstabe stellt einen der 16 Tänzer da.

Es wird auch direkt die von allen Tänzern zurückgelegte Distanz *sum_dancer_overall_distance* berechnet, darauf gehe ich aber erst in Sonderfälle ein.

Es gibt auch__hash__() in der Klasse, diese Sondermethode ermöglich das Hashen von den Objekten von der Klasse Figuren, und so das Einfügen in Sets.

2.2 Ausführen einer Figur

execute_routine() ist eine Methode, die eine Figur und Anfangsposition als Parameter. Diese führt die Figur aus und gibt die Aufstellung zurück. Dabei wird über jeden Tänzer von *pos_after* von der auszuführende Figur iteriert. Für jeden Tänzer wird dann ermittelt von welcher Position er kommt, indem man die Position im Alphabet hernimmt. Das ist möglich, weil die *pos_after* einer Figur die Stellung nach dem Ausführen der Figur mit ABCDEFGHIJKLMNOP ist. Diese Liste ist alphabetisch geordnet, es steht jeder Buchstabe an seiner Position im Alphabet startend bei 0 für A. Nachdem man weiß von welcher Position sich ein Tänzer herbewegt, kann man diesen in *initia_pos* suchen und in *new_pos* einfügen. *new_pos* wird dann zurückgegeben.

Im Folgendem Beispiel wird dies mit 6 Tänzern dargestellt. Es wird der Alphabet-Index von *pos_ after* erstellt und anschließend damit die neue Reihenfolge von *new_ pos* aus *intial_ pos* gebildet.

Tabelle 1: Beispiel: Ableitung der neuen Stellung aus pos_after

pos_after	A	C	B	F	E	D
Alphabet-Index	0	2	1	5	4	3

Tabelle 2: Bildung der Neuen Stellung aus zufälliger Anfangsstellung (initial_pos)

initial_pos	B	A	C	D	F	E
new_pos	B	C	A	E	F	D

2.3 Rekursive Suche aller Choreographien

Die rekursive Suche aller gültigen Choreographien erfolgt in *find_ choreos()*. Darin wird einmal die Liste aller Figuren und die Dauer des Musikstücks als Parameter eingegeben. Diese Funktion speichert aber nur die gültigen Choreographie, damit sie bei dem Aufruf von *search()*, also der rekursiven Funktion nicht immer neu leer initialisiert wird. In *search()* werden also die Kombinationen rekursiv gebildet. *search()* wird nur einmal von *find_ choreos()* mit der geordneten Startposition, einer leeren Liste, zu der dann Figuren hinzugefügt werden um eine Choreographie zu bilden, und die Länge des Musikstückes abgerufen. Es wird dann jeweils durch die Liste aller Figuren iteriert, für jede Iteration wird die jeweilige Figur der Choreographie angefügt und dasselbe mit der aktualisierten Rest-Summe, also wie viele Takte in der Choreographie noch frei sind und der neuen letzten Stellung wiederholt.

Falls die Restdauer einer Kombination 0 ist, heißt das, dass die Länge der Choreographie die des Musikstückes entspricht. Wenn die Endposition gleich der Startposition ist, ist die Choreographie gültig und wird in der Liste der gültigen Choreographien gespeichert.

Falls die Restdauer negativ ist, ist die Kombination an Figuren zu lang und wird verworfen.

Nachdem die Suche immer fertig ist, wird die letzte Figur der Kombinaion entfernt und durch die Iteration der Figurenliste mit einer neuen ersetzt. Dies wird gemacht, bis es keine Kombination mehr gibt, die unerforscht ist. Alle gültigen Kombinationen sind dann in *legal_ choreos* gespeichert.

2.4 Verkürzung der Laufzeit

Nachdem ich meinen fertigen Code an den Beispielen der BWINF-Webseite angewendet habe, habe ich gemerkt, dass die Findung aller gültigen Choreographien bei dem Beispiel choreo04 eine Ewigkeit dauert. Ich habe die Laufzeit dadurch verkürzt, dass ähnliche Kombinationen an Figuren während der Suche nur einmal durchsucht werden. Dies wurde gemacht, indem ich alle gesehenen Zustände speichere. Ein Zustand beinhält Länge, genutzte Figuren, Restdauer und Anordnung der Tänzer. Falls ein Zustand schon gefunden und so durchsucht wurde, bedeutet das, dass man man den anderen nicht mehr durchsuchen muss, da er auf dasselbe hinausführt. Es wird aber gespeichert, ob schon die Suche einer Choreographie deswegen aufgegeben wurde, dies braucht man, damit nicht ausgegeben wird, dass es nur eine mögliche Kombination gibt. Dies speichert *single_ choreo*. Die Laufzeit von Beispiel 4 beträgt aber immer noch 7.7 Minuten. Außerdem werden fast 12 GB RAM genutzt.

2.5 Sonderfälle

Für jeden Sonderfall wird jeweils eine Funktion aufgerufen, die die beste Choreographie nach einem Kriterium und den besten Wert ausgibt. Darin wird der beste Wert ermittelt, und dann die Liste der Choreographien nach einer Choreographie durchsucht, die diesen Wert hat.

möglichst viele verschiedene Figuren

Jede Choreographie wird erst zu einem Set gemacht, und dann wieder in eine Liste, um gleiche Figuren nur einmal zu beinhalten. Der längste Set ist der beste Wert, da es die meisten verschiedenen Figuren gibt.

möglichst viele Figuren

Die längste Liste für eine Choreographie wird ausgewählt.

möglichst viele Figuren

Die kürzeste Liste für eine Choreographie wird ausgewählt.

Möglichst kleine Strecke für alle Tänzer

Es wird bei der Erstellung einer Figur in der Klasse Routine direkt die Distanz von allen Tänzern berechnet, die sie bei der Ausführung zurücklegen. Für jeden Tänzer wird die Differenz von der aktuellen Position und der alten Position, also der Stelle im Alphabet, zusammengezählt. Für jede Choreographie wird der Wert für jede Figur zusammengezählt. Der kleinste Wert davon ist der beste.

Möglichst große Strecke für alle Tänzer

Es wird genau dasselbe, wie bei der kleinsten Strecke gemacht, nur dass der größte Wert der beste ist.

2.6 Ausführung

Die Ausführung aus dem Computer-Terminal erfolgt so, dass man sich erst im Verzeichnis des Programms A2.py befinden muss. Das Programm wird durch den Befehl *python A2.py eingabedatei.txt* gestartet. *eingabedatei.txt* wird dabei mit dem Pfad der Datei ersetzt, in der das Musikstück und die Figuren in Noras Notation niedergeschrieben sind.

Nachdem die Figuren aus der Eingabedatei in Objekte der Klasse Routine umgewandelt werden, kann man alle gültigen Choreographien suchen. Nachdem diese gefunden wurden wird geprüft, ob es nur eine oder keine gültige Choreographie gibt, falls das zutrifft wird das ausgegeben, ansonsten werden die besten Choreographien zu den Kriterien gesucht und ausgegeben.

3 Werkzeuge

Es wurde keine KI bei der Ideengewinnung oder bei der Umsetzung benutzt, ich habe jedoch die kostenlose ChatGPT-Version ausschließlich zur Überprüfung des Codes auf Leichtsinnsfehler benutzt. Bei Unklarheiten, wie etwas in Python implementiert wird, wie z.B. Sets innerhalb von Sets habe ich Hilfe bei StackOverflow gefunden.

Um die Terminalargumente zu implementieren habe ich die Library sys in python importiert. *sys.argv[1]* nimmt dann die nächste Terminaleingabe nach der Ausführungsdatei, die dann in meinem Code geöffnet wird.

4 Beispiele

Es werden die Ausgaben von dem Programm beim Ausführen mit den BWINF-Beispielen dokumentiert. Bei machen sind die Ausgaben sehr lang, da alle Figuren von einer Choreographie jeweils aufgelistet werden. Bei denen wird nur der beste Wert gezeigt und nicht die ganze Auflistung der Figuren. Die ganze Ausgabe befindet sich dann in dem Ordner *A2_Ausgaben* als .txt-Datei und enthalten den Namen der Eingabedatei.

choreo01.txt

- Choreo mit möglichst viele unterschiedliche Figuren: 2
- Choreo mit möglichst viele Figuren: 16
- Choreo mit möglichst wenige Figuren: 12
- Choreo mit von allen Tanzenden zurückgelegte Distanz möglichst groß: 384
- Choreo mit von allen Tanzenden zurückgelegte Distanz möglichst klein: 312

choreo02.txt

Keine gültigen Choreographien gefunden

choreo03.txt

Keine gültigen Choreographien gefunden

choreo04.txt

- Choreo mit möglichst viele unterschiedliche Figuren: 6
- Choreo mit möglichst viele Figuren: 12
- Choreo mit möglichst wenige Figuren: 12
- Choreo mit von allen Tanzenden zurückgelegte Distanz möglichst groß: 1154
- Choreo mit von allen Tanzenden zurückgelegte Distanz möglichst klein: 1154

choreo05.txt

- Choreo mit möglichst viele unterschiedliche Figuren: 2
- Choreo mit möglichst viele Figuren: 24
- Choreo mit möglichst wenige Figuren: 6
- Choreo mit von allen Tanzenden zurückgelegte Distanz möglichst groß: 1728
- Choreo mit von allen Tanzenden zurückgelegte Distanz möglichst klein: 504

choreo06.txt

- Choreo mit möglichst viele unterschiedliche Figuren: 4
- Choreo mit möglichst viele Figuren: 16
- Choreo mit möglichst wenige Figuren: 4
- Choreo mit von allen Tanzenden zurückgelegte Distanz möglichst groß: 768
- Choreo mit von allen Tanzenden zurückgelegte Distanz möglichst klein: 256

choreoA.txt

Keine gültigen Choreographien gefunden

5 Quellcode

5.1 Klasse für Figuren

```
class Routine:
    """Klasse fuer Bewegungen in einer Choreografie mit Name, Dauer und Position nach der Ausfuehrung
    beginnend mit ABCDEFGHIKLMNOP"""

    def __init__(self, name, duration : int, pos_after):
        self.name = name
        self.duration = duration
        self.pos_after = pos_after

        # Summe von allen Taenzern zurueckgelegten Strecke:
        # Differenz zwischen aktueller Position (Index)
        und urspruenglicher Startposition (ord(char.lower()) - 97)
        # ord(char.lower()) - 97 gibt Position im Alphabet zurueck: a = 0, b = 1, c = 2 ...
        self.sum_dancer_overall_distance =
            sum([abs(index - (ord(char.lower()) - 97)) for index, char in enumerate(pos_after)])
```

```
    def __hash__(self): # damit man Routine in ein Set hinzufuegen darf
        return hash((self.name, self.duration, self.pos_after))
```

5.2 Ausführen einer Figur

```
    def execute_routine(routine, initial_pos):
    """Gibt Position nach Ausfuehrung der Routine ab initial_pos zurueck"""
    new_pos = ""
    for dancer in routine.pos_after:
        # ord(char.lower()) - 97 gibt Position im Alphabet zurueck: a = 0, b = 1, c = 2 ...
        new_pos += initial_pos[ord(dancer.lower()) - 97] # new_pos ist ein String;
        # jeder Taenzer (Buchstabe) wird zum neuen Platz in der richtigen Reihenfolge
        # dazu konkateniert (Fachbegriffe sind immer toll :))
    return new_pos
```

5.3 rekursive Suche aller Choreographien

```
    def find_choreos(routines, choreo_length):
    seen = set()
    legal_choreos = []
    single_choreo = True

    def search(current_position, rest_sum, combination):
        nonlocal single_choreo

        if rest_sum < 0:
            return

        if rest_sum == 0:
            if current_position == starting_position:
                legal_choreos.append(list(combination))
            return

        state = (current_position, rest_sum, len(combination), frozenset(combination))
        if state in seen:
            single_choreo = False
            return
        seen.add(state)

        for routine in routines:
            combination.append(routine)
            new_position = execute_routine(routine, current_position)
            search(new_position, rest_sum - routine.duration, combination)
            combination.pop()

    search(starting_position, choreo_length, [])
    return legal_choreos, single_choreo
```

5.4 Sonderfälle

```
def max_different_routines(choreos):
    """Sucht nach Choreos mit den meisten unterschiedlichen Routinen und gibt eine zurueck"""

    best_value = max(len(list(set(choreo))) for choreo in choreos)
    # list(set()), um doppelte Routinen zu entfernen,
    print(best_value)
    for choreo in choreos:
        if len(list(set(choreo))) == best_value:
            return choreo
    return None
```

```
def max_routines(choreos):
    """Sucht nach Choreos mit den meisten Routinen und gibt eine zurueck"""

    best_value = max(len(choreo) for choreo in choreos)
    print(best_value)
    for choreo in choreos:
        if len(choreo) == best_value:
            return choreo
    return None

def min_routines(choreos):
    """Sucht nach Choreo mit den wenigsten Routinen und gibt eine zurueck"""

    best_value = min(len(choreo) for choreo in choreos)
    print(best_value)
    for choreo in choreos:
        if len(choreo) == best_value:
            return choreo
    return None

def max_distance(choreos):
    """Sucht nach Choreo mit der groessten von den Taenzern
    zurueckgelegten Strecke und gibt eine zurueck"""

    # Maximum aller Summen der von Taenzern zurueckgelegten Strecken in jeder gueltigen Choreo
    best_value = max(sum(routine.sum_dancer_overall_distance for routine in choreo)
        for choreo in choreos)
    print(best_value)

    for choreo in choreos:
        if sum(routine.sum_dancer_overall_distance for routine in choreo) == best_value:
            return choreo
    return None

def min_distance(choreos):
    """Sucht nach Choreo mit der kleinsten von den Taenzern
    zurueckgelegten Strecke und gibt eine zurueck"""

    # Minimum aller Summen der von Taenzern zurueckgelegten Strecken in jeder gueltigen Choreo
    best_value = min(sum(routine.sum_dancer_overall_distance for routine in choreo)
        for choreo in choreos)
    print(best_value)

    for choreo in choreos:
        if sum(routine.sum_dancer_overall_distance for routine in choreo) == best_value:
            return choreo
    return None
```

5.5 Ausführen des Programmes

```
valid_choreos, only_one_choreo = find_choreos(routines_piece_of_music, piece_length)

#[...]

    if len(valid_choreos) == 0:
        #Keine gueltige Choreo gefunden
    elif len(valid_choreos) == 1 and only_one_choreo == True:
        #Eine gueltige Choreo gefunden
    else:
        #Aufzaehlung der Sonderfaelle
```